RNN 作业报告

深度学习

学　　号 2354369

姓　　名 陈宇轩

专　　业 基础学科拔尖基地（计算机）

授课老师 程大伟

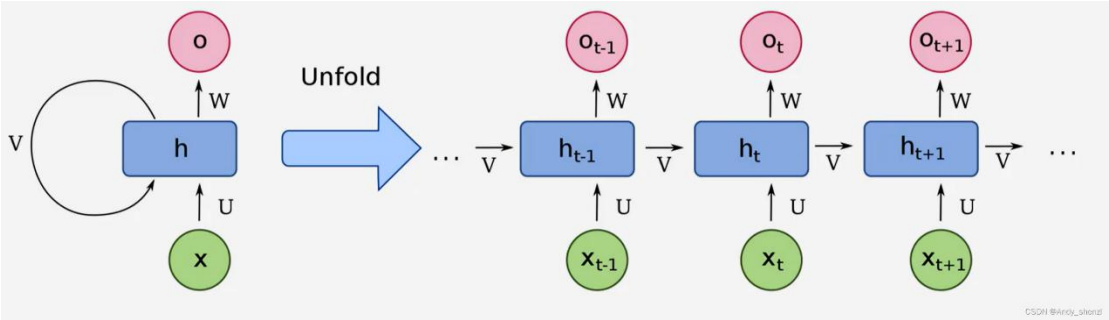
目录

一. 模型解释.....3
1.1 RNN.....3
1.2 LSTM.....3
1.3 GRU.....4
二. 诗歌生成过程.....5
2.1 数据预处理阶段.....5
2.2 模型构建阶段.....5
2.3 训练阶段.....5
2.4 生成阶段.....6
三. 实验结果.....6
3.1 训练过程.....6
3.2 生成诗歌结果.....7
3.3 总结.....8

一. 模型解释

1.1 RNN

RNN(Recurrent Neural Network)是一类专门用于处理序列数据的神经网络。与传统前馈神经网络不同，RNN 在隐藏层之间引入了循环连接，使得网络能够保持对之前输入的“记忆”。



可以看到，网络除了输入输出以外，还存在隐状态 h，这是 RNN 的特点。当前时刻的隐状态依赖于上一时刻的隐状态，这种隐状态传递特性使得 RNN 具有处理序列的能力。

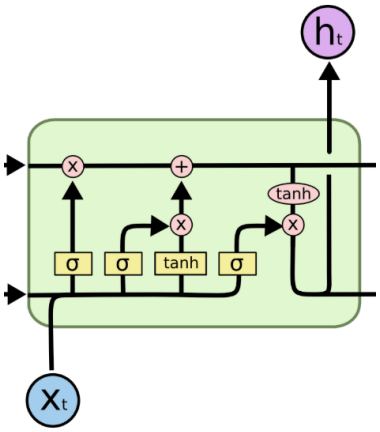
公式可以总结如下：

$$h_t = tanh(Ux_t + Vh_{t-1} + b)$$

$$O_t = Wh_t + b$$

1.2 LSTM

LSTM(Long Short-Term Memory)是 RNN 的一种改进变体,通过引入门控机制和细胞状态解决标准 RNN 的长期依赖问题。



核心组件包括：

1. 遗忘门：决定从细胞状态中丢弃什么信息。公式：

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

2. 输入门：决定将什么新信息存储到细胞状态中。公式：

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_f)$$

3. 候选细胞状态：创建候选值向量，用于更新细胞状态。公式：

$$\tilde{C}_t = tanh(W_c \cdot [h_{t-1}, x_t] + b_C)$$

4. 细胞状态更新：更新细胞状态。公式：

$$C_t = f_t \cdot C_{t-1} + i_t \cdot \tilde{C}_t$$

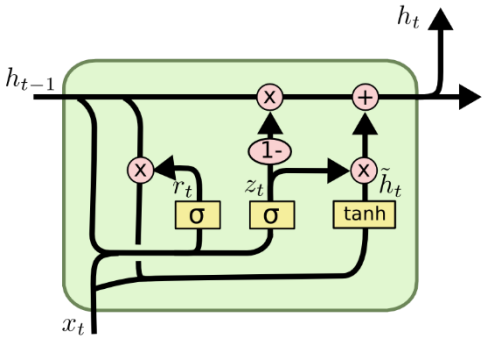
5. 输出门。决定输出什么值。公式：

$$O_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_O)$$

$$h_t = O_t \cdot tanh(C_t)$$

1.3 GRU

GRU(Gated Recurrent Unit)是 LSTM 的简化版本，于 2014 年提出。它将 LSTM 的遗忘门和输入门合并为“更新门”,同时合并了细胞状态和隐状态。



$$z_t = \sigma\left(W_z \cdot [h_{t-1}, x_t]\right)$$
$$r_t = \sigma\left(W_r \cdot [h_{t-1}, x_t]\right)$$
$$\tilde{h}_t = \tanh\left(W \cdot [r_t * h_{t-1}, x_t]\right)$$
$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$$

核心组件包括：

1. 重置门：决定在计算新的候选隐状态时，如何使用过去的隐状态。公式：

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t] + b_r)$$

2. 更新门：决定当前时间步的隐状态应保留多少过去的隐状态，以及应添加多少新信息。公式：

$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t] + b_z)$$

3. 候选隐状态：公式：

$$\tilde{h}_t = tanh(W_h \cdot [r_t \cdot h_{t-1}, x_t] + b_h)$$

4. 隐状态更新：公式：

$$h_t = (1 - z_t) \cdot h_{t-1} + z_t \cdot \tilde{h}_t$$

相较 LSTM，GRU 只有 2 个门，而 LSTM 有 3 个门，在保持相当的性能的同时，做到计算效率更高、结构更简洁、训练更容易。

二. 诗歌生成过程

2.1 数据预处理阶段

该阶段下，首先要读取唐诗数据：从 poems.txt 文件中读取唐诗数据，每行格式为“标题：内容”。再提取诗歌内容部分。

接着，要添加特殊标记。在每首诗开头添加 G 标记，在每首诗结尾添加 E 标记。进行简单的数据过滤，过滤掉过长(>80 字)或过短的诗歌，过滤掉包含特殊字符的诗歌。

再次要构建词汇表。统计所有字符的出现频率，按频率降序排序。建立 word_to_int(字符到 ID)和 int_to_word(ID 到字符)映射字典，词汇表大小约 6100 个字符。

最后进行序列编码。将每首诗的字符序列转换为对应的 ID 序列，记录每首诗的序列长度。

2.2 模型构建阶段

模型架构分为输入层、嵌入层、LSTM、全连接层、输出层五层。输入层输入字符 ID，在嵌入层将离散的 ID 转换为稠密向量，2 层 LSTM 处理序列信息，提取上下文特征，全连接层输出每个字符的概率分布，输出层预测下一个字符

关键参数有：词汇表大小 6123，嵌入维度 100，LSTM 隐藏单元数 128，LSTM 层数，Batch size 430。

2.3 训练阶段

该阶段首先准备训练数据，将诗歌序列输入 x 和目标输出 y 配对：$t_{n-1} - t_n$，即下一位输入字符即为目标输出。使用 padding 将同一 batch 内的序列填充到相同长度。

第二步前向传播。将输入序列送入模型，模型输出每个位置的 logits(每个词汇的概率分布)。

第三步计算损失。损失函数使用负对数似然损失(NLLLoss)来计算预测分布与真实下一个字符的差异。

第四步反向传播和优化。使用 RMSprop 优化器(学习率 0.01，后降到 0.003)计算梯度并更新模型参数，使用梯度裁剪(clip_grad_norm)防止梯度爆炸

第五步迭代训练。初始训练 400 个 batch (约 1 个 epoch)，继续训练额外 9 个 epoch。Loss 从 8.72(epoch = 0, batch = 0)下降到约 5.5。

2.4 生成阶段

Gen_poems.py 完成该阶段任务。利用预训练模型，该程序可以按如下算法生成诗歌：

1. 输入：指定的开头字符(如“日”)
2. 初始化 LSTM 的隐状态和细胞状态
3. 循环(最多 50 次或遇到结束符):
    a. 将当前字符序列转换为 ID
    b. 通过嵌入层得到字符向量
    c. 通过 LSTM 层更新隐状态
    d. 通过全连接层输出所有词汇的概率
    e. 选择概率最高的字符作为下一个字符
    f. 如果是结束符(E),停止生成
    g. 将生成的字符添加到诗歌中
4. 输出：生成的诗歌

三. 实验结果

3.1 训练过程

总诗歌数：43,030 首

词汇表大小：6,123 个字符

Batch size：100

每个 epoch：430 个 batch

训练配置：

优化器：RMSprop

学习率：0.01 (初期) → 0.003 (继续训练)

损失函数：NLLLoss

梯度裁剪：max_norm = 1

```
******************************
epoch   0 batch number 0 loss is:  8.717989921569824
finish  save model
prediction [1080, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3]
b_y        [267, 2888, 267, 2045, 0, 26, 1067, 26, 349, 1, 3, 3]
******************************
epoch   0 batch number 1 loss is:  7.602354526519775
prediction [540, 275, 77, 534, 275, 317, 275, 317, 317, 317, 275, 275, 349, 5950]
b_y        [617, 564, 80, 601, 132, 0, 9, 184, 2007, 373, 305, 1, 3, 3]
******************************
epoch   0 batch number 2 loss is:  8.419548034667969
prediction [20, 20, 20, 20, 20, 20, 20, 20, 20, 20, 20, 20, 20, 20, 20, 20, 20, 20]
b_y        [125, 9, 3862, 695, 16, 1721, 1721, 0, 373, 591, 246, 695, 900, 80, 431, 1, 3, 3]
******************************
epoch   0 batch number 3 loss is:  7.578549385070801
prediction [0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
b_y        [1539, 1058, 3996, 64, 0, 793, 977, 550, 381, 1, 3867, 3285, 972, 1287, 0, 1541, 1541
******************************
epoch   0 batch number 4 loss is:  7.373046398162842
prediction [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 3, 3, 3, 3]
b_y        [194, 348, 66, 89, 73, 0, 574, 21, 855, 101, 1013, 1, 293, 162, 1283, 117, 383, 0, 25
******************************
epoch   0 batch number 5 loss is:  8.345693588256836
prediction [7, 7, 51, 35, 35, 35, 35, 35, 35, 35, 35, 35, 35, 35, 35, 35, 35, 35, 35, 35, 35, 35
b_y        [349, 351, 456, 220, 850, 0, 304, 146, 195, 350, 576, 1, 264, 178, 35, 323, 189, 0, 2
******************************
epoch   0 batch number 6 loss is:  9.300287246704102
prediction [28, 7, 16, 16, 16, 16, 15, 16, 16, 16, 16, 16, 16, 16, 16, 16, 16, 16, 16, 16, 16, 1
b_y        [18, 517, 56, 17, 191, 0, 1552, 1231, 88, 298, 61, 1, 57, 252, 311, 129, 259, 0, 397,
******************************
epoch   0 batch number 7 loss is:  8.05212688446045
prediction [182, 23, 23, 17, 17, 17, 17, 17, 17, 17, 17, 17, 17, 17, 17, 17, 17, 17, 17, 17, 17,
b_y        [3049, 18, 738, 137, 60, 0, 60, 127, 375, 36, 8, 1, 742, 20, 236, 117, 30, 0, 331, 16
******************************
epoch   0 batch number 8 loss is:  7.069976329803467
```

用 Loss 下降体现寻来你过程：从 Epoch 0, Batch 0：Loss = 8.718，经过浮动和下降，最终稳定在 5.5 附近。

3.2 生成诗歌结果

初期训练（第 1 个 epoch）大部分诗歌只生成 1 个字，且不连贯。继续训练后可以生成更长、更连贯的句子。生成结果如下：

```
以'日'开头的诗:
日西来路，何处空山水。何处无人事，何人更有情。
------------------------------

以'红'开头的诗:
红上，春风吹雨发。风声不可见，不觉有人间。不是无人事，何如此路难。
------------------------------

以'山'开头的诗:
山
------------------------------

以'夜'开头的诗:
夜华中光里，春风吹落照。何时不可见，不觉有谁同。
------------------------------

以'湖'开头的诗:
湖西水夜，不觉春风起。何事不知处，不知何处人。
------------------------------

以'海'开头的诗:
海
------------------------------

以'月'开头的诗:
月前山上，春风吹雨发。何当不得意，不觉有谁同。
以'山'开头的诗:
山
------------------------------
```

3.3 总结

本次作业中，可以看出生成质量主要由以下因素决定：训练时间：训练 epoch 数过少会导致欠拟合，在本次作业中基本呈现训练 epoch 数增加，训练效果变好的趋势。模型容量：更大的隐藏层维度和更深的 RNN 层数对性能有提升。数据量：数据集提供了四万余首诗，数据丰富、格式统一。生成策略：本次选用了选择概率最高的词的算法，不同的算法会导致不同的结果。

可能可以通过以下方法改进：使用 Transformer 替代 LSTM，引入注意力机制，可以捕捉长距离依赖，提升生成质量；可以引入格律、韵脚等先验信息；使用学习率衰减策略，并引入验证集防止过拟合。